



COPILOTTRAINING
TECHNICAL BRIEFING

Useful Autonomy, Clear Authority

| GitHub Copilot governance from tool request to protected merge

Clear decision rights let routine work move and consequential changes reach the right owner.

Practitioner tech talk · Verify and Govern



"How can a platform team let useful Copilot work move while keeping authority explicit?"

Follow one authentication-middleware change from local execution to protected merge.

Every boundary returns a decision, evidence, or an owner route.

Developer

Completes bounded implementation work with visible routing decisions



Platform team

Places each decision at the layer that can enforce it



Service owner

Accepts protected changes through repository controls



One change

Authentication middleware stays in view from edit to merge

Three routes

Allow, ask, and deny each produce an actionable result

Five homes

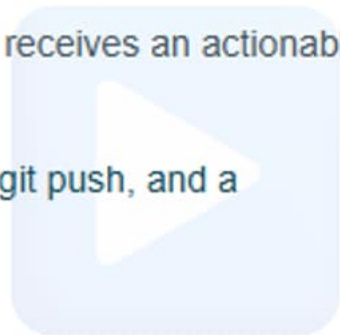
Availability, context, execution, verification, and approval

PART 1

Clear Runtime Routes

Every Copilot tool request receives an actionable decision

Compare a routine edit, a git push, and a protected policy edit

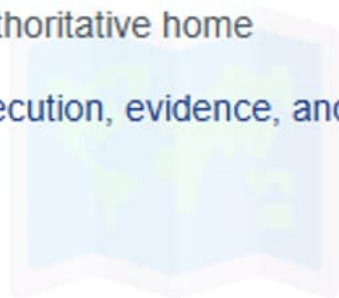


PART 2

Decision Rights

Each decision has one authoritative home

Place access, context, execution, evidence, and acceptance

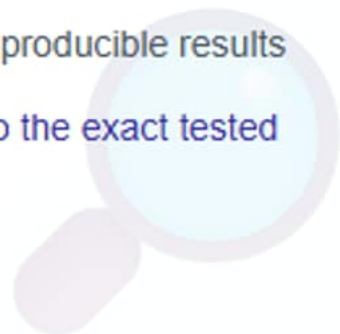


PART 3

Reviewable Evidence

The pull request carries reproducible results

Connect delegated work to the exact tested revision



PART 4

Durable Acceptance

Qualified acceptance survives the local session

Trace checks, ownership, protection, and bypass authority



Every Tool Request Gets a Clear Route

Copilot CLI proposes an authentication-middleware action. The repository hook evaluates it before execution.



Allow Copilot

Make the routine middleware edit



Ask the developer

Confirm before Copilot pushes



Route to the owner

Direct protected policy to the authorized team

Three separate Copilot requests

↳ edit → allow | git push → ask | policy edit → deny

The Repository Hook Establishes the Runtime Boundary

.github/hooks/governance.json

```
{
  "hooks": {
    "PreToolUse": [{
      "type": "command",
      "command": "node scripts/pre-tool-use.mjs",
      "timeout": 5
    }]
  }
}
```



Copilot requests a tool

PreToolUse sends the pending action to the hook as JSON



The hook evaluates it

The repository script applies the team policy to that request



The tool waits

The hook has five seconds to return allow, ask, or deny

✓ Copilot receives a policy decision before its requested tool executes.

Three Runtime Routes Keep Development Moving Safely

scripts/pre-tool-use.mjs

```
if (matchesSensitivePath(values))  
  return decision("deny", ownerLedRoute)  
  
if (matchesPublicationCommand(values))  
  return decision("ask", confirmationReason)  
  
return decision("allow", workspaceReason)
```

✓ Allow Copilot

A routine middleware edit continues automatically

? Ask the developer

A git push waits for explicit human confirmation

→ Route to the owner

A protected policy edit moves to the authorized team

Deterministic Fixtures Prove All Three Runtime Boundaries

Condensed PreToolUse payloads · 3/3 verified

IN tool_name=edit path=src/auth/middleware.ts
OUT hookSpecificOutput.permissionDecision="allow"

IN tool_name=shell command=git push origin feature/auth-timeout
OUT hookSpecificOutput.permissionDecision="ask"

IN tool_name=edit path=src/auth/policy/production.json
OUT hookSpecificOutput.permissionDecision="deny"

✓ Middleware edit

Copilot continues because the file is inside the agreed boundary

? Git push

Copilot pauses because publication changes repository state

→ Policy edit

Copilot stops because the protected path requires its owner

✓ Allow, ask, and deny are alternative answers to separate requests, not steps in a sequence.

Every Boundary Returns a Visible Result and Recovery Route



Unexpected event

Trigger

The payload is not a PreToolUse event

Result

permissionDecision=deny with an explicit reason

Recovery

Verify hook registration and payload before retrying



Malformed input

Trigger

The hook cannot parse the incoming JSON

Result

The process writes the parsing error and exits nonzero

Recovery

Repair the producer or configuration, then rerun the fixture



Protected request

Trigger

Copilot requests a change under src/auth/policy/

Result

permissionDecision=deny names the protected boundary

Recovery

Split the task and route policy work to its service owner



Uncertainty becomes a visible stop; protected work gets an owner-led route.

Each Decision Has an Authoritative Home

Enterprise policy grants access. Repository files bound execution. Rulesets enforce merge authority.



Enterprise settings

Copilot policies control CLI and MCP availability



Repository .github/

Instructions, MCP config, hooks, workflows, and owners travel with code



Repository rules

Settings → Rules → Rulesets enforces protected merge

One Copilot change

↳ hosted policy → repo files → hosted enforcement

Assign Each Operating Decision to Its Authority Surface

Follow each decision to its hosted setting, repository file, or rule

Use CLI?	Copilot → Policies enables or disables Copilot CLI	hosted setting
-----------------	--	----------------

Use what?	.github/copilot-instructions.md supplies repository guidance	repo file
------------------	--	-----------

Use MCP?	MCP policy enables servers; managed settings allow or deny them	enterprise policy
-----------------	---	-------------------

Run tool?	governance.json runs pre-tool-use.mjs before execution	repo hook
------------------	--	-----------

Passed?	change-evidence.yml publishes the required check	GitHub Actions
----------------	--	----------------

Merge?	CODEOWNERS plus Settings → Rules requires acceptance	hosted rule
---------------	--	-------------

Enterprise Policy Enables Approved MCP Capabilities

governance/mcp-managed-settings.json

```
{
  "allowedMcpServers": [
    { "serverUrl": "https://mcp.example.com/*" }
  ],
  "deniedMcpServers": [
    { "serverCommand": [
      "npx", "-y",
      "@modelcontextprotocol/server-filesystem", "/"
    ] }
  ]
}
```

↔ Enable MCP

Copilot → Policies → MCP must permit MCP servers

✓ Match an allow rule

Approved URLs or commands may start; unmatched servers are blocked

⊖ Deny wins

A denied server stays blocked even when an allow rule matches

💡 Registries support discovery. Managed allow and deny settings enforce access.

The Repository Requests Tools; Enterprise Policy Authorizes Access

.github/mcp.json + documented policy result

```
DECLARED  work-items  
          type=http  url=https://mcp.example.com/work-items  
          tools=[get_issue, search_docs]
```

```
EVALUATED work-items      → ALLOWED (URL matches allowlist)  
          filesystem-root → BLOCKED (command matches denylist)
```



Repository declaration

.github/mcp.json shares the server definition and selected tools



Folder trust

Copilot CLI loads workspace servers only from a trusted folder



Enterprise evaluation

The CLI applies managed allow and deny rules before connecting

✓ Approval stays bounded by selected tools, credentials, and service permissions.

Repository Files Declare Context, Tools, and Runtime Rules

Repository control surfaces

```
.github/  
├─ copilot-instructions.md  
├─ mcp.json  
├─ hooks/governance.json  
├─ workflows/change-evidence.yml  
├─ CODEOWNERS  
└─ rulesets/protected-main.json
```

```
scripts/pre-tool-use.mjs
```

Guide + declare

Instructions guide Copilot;
mcp.json proposes shared
server tools

Route

governance.json runs pre-
tool-use.mjs to evaluate
each pending tool request

Verify

change-evidence.yml runs
tests, lint, and dependency
review on the pull request

Approve

CODEOWNERS names the
reviewer; the ruleset
requires that review and
check

Each Layer Owns a Specific Decision Boundary



Enterprise policy

Owns Copilot CLI and MCP server availability; repository acceptance stays downstream



Instructions + hook

Guides Copilot and routes pending tool requests; direct terminal commands remain human-driven



Workflow + ruleset

Requires pull-request evidence and authorized review before GitHub accepts the merge

✓ Use a sandbox or direct human authority when consequences extend beyond these enforcement points.

Delegated Work Carries Evidence

Repository guidance shapes the middleware change; automated checks attach reproducible results to its commit.



Guidance

Intent and edge cases stay visible



Checks

Results are reproducible



Review

Advice stays distinct from acceptance

Middleware pull request

↳ artifact + checks → reviewable evidence

Custom Instructions Anchor Output in Repository Standards

Repository baseline + path-scoped auth guidance

```
.github/copilot-instructions.md
- Use Node.js 22 and the committed npm lockfile.
- Run focused tests and npm run lint.
- Report commands, outcomes, and unresolved findings.

.github/instructions/auth.instructions.md
applyTo: src/auth/**
- Preserve deny-by-default authentication.
- Test invalid credentials, timeouts, and downstream failure.
- Run npm test -- --runInBand test/auth.
```



Repository baseline

Stable runtime, structure, and validation guidance applies everywhere



Auth-only contract

The applyTo selector adds trust-boundary and test guidance only under src/auth



Enforcement stays separate

Hooks gate tools; CODEOWNERS and rulesets preserve acceptance authority

✓ Instructions guide Copilot. Executable controls enforce consequential boundaries.

Automated Checks Attach Reproducible Proof to the Commit

.github/workflows/change-evidence.yml

```
permissions:
  contents: read

steps:
  - run: npm ci
  - name: Focused authentication tests
    run: npm test -- --runInBand test/auth
  - name: Lint
    run: npm run lint
  - name: Dependency review
    uses: actions/dependency-review-action@v4
```



Focused tests

Authentication behavior gets a dedicated test path



Lint

Repository quality rules publish their result



Dependencies

Dependency review adds an independent signal

Machine Evidence Informs; the Service Owner Accepts



Advisory findings

Source

Copilot code review

Question

What deserves human attention?

Result

Review comments or findings; no merge authority



Machine evidence

Source

Actions, code scanning, and secret scanning

Question

What did automation observe on this commit?

Result

Checks and findings tied to the tested revision



Service-owner approval

Source

The matching authentication code owner

Question

Is this evidence sufficient for acceptance?

Result

Approve or request changes; the ruleset enforces it



Tools publish observations. The service owner makes the acceptance decision.

Merge Authority Outlasts the Session

CODEOWNERS and rulesets carry acceptance authority beyond the local Copilot session.



Checks

Required evidence must pass



Approval

Qualified ownership is explicit



Protection

Bypass and force-push stay governed

Protected branch

↳ evidence + owner approval → merge

Code Ownership Routes Review to the Authorized Team

.github/CODEOWNERS + pull-request result

```
/src/auth/ @octo-org/auth-service-owners
/test/auth/ @octo-org/auth-service-owners
.github/workflows/ @octo-org/platform-security
.github/CODEOWNERS @octo-org/platform-security
```

```
CHANGED src/auth/middleware.ts
MATCHED /src/auth/
REQUEST @octo-org/auth-service-owners
```

Path match

GitHub compares each changed file with the CODEOWNERS patterns

Review request

The matching team is requested because it owns that part of the repository

Enforcement handoff

CODEOWNERS names the reviewer; the ruleset on slide 21 requires their approval

✓ Ownership selects the reviewer. The ruleset turns that selection into a merge condition.

Rulesets Recheck Every Acceptance Condition at Merge

Protected default branch · ruleset evaluation

TARGET	~DEFAULT_BRANCH	ENFORCEMENT	active
REQUIRE	code-owner approval	✓	auth owner approved
REQUIRE	approval after the latest push	✗	new commit needs review
REQUIRE	resolved review threads	✗	1 conversation open
REQUIRE	current change-evidence check	✓	passed on latest commit
BYPASS	none		
RESULT	MERGE BLOCKED		

Target + enforcement

The active ruleset evaluates every change headed for the default branch

Qualified approval

An approval counts only when the matching code owner supplies it

Fresh after each push

A later commit can require new approval and a current status check

Explicit bypass

An empty bypass list leaves no actor outside these merge conditions

 A passing check is insufficient by itself: every required condition must be current at merge time.

Repository Rules Preserve Authority Beyond the Local Session



Every entry path

GitHub evaluates pushes from local Git, the web, APIs, and automation against the hosted ruleset



Branch deletion blocked

The deletion rule prevents removal of the protected default branch



History rewrite blocked

The non-fast-forward rule rejects force-pushes that replace accepted history



New commits reopen the gate

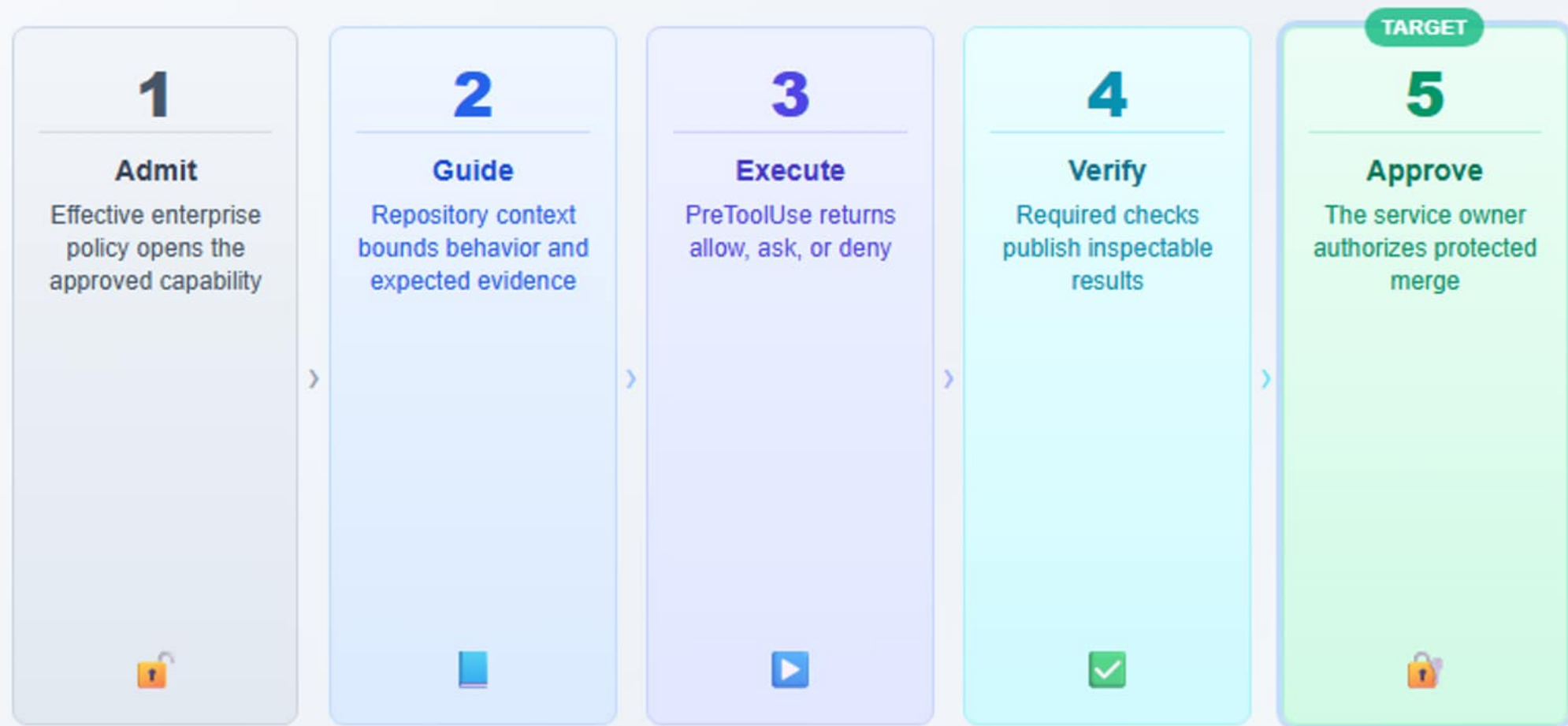
A later push must satisfy the configured approval and evidence conditions again



Hooks govern Copilot tool requests. Rulesets govern what GitHub accepts.

Follow the Authority Chain from Request to Merge

Each stage contributes a distinct decision and observable result



The developer produces the change and evidence; the service owner accepts or requests another iteration.

Clear Authority Makes Confident Delegation Possible



Routine work keeps moving

PreToolUse allows bounded edits, pauses publication, and routes protected work to its owner



External tools stay bounded

Enterprise policy authorizes repository-declared MCP servers, tools, and credentials



Evidence follows the revision

Tests, scans, and status checks describe the exact commit awaiting review



Authority survives the session

CODEOWNERS and rulesets require qualified approval across every GitHub entry path

💡 Every consequential decision has an owner, an enforcement point, observable evidence, and a recovery route.

Prove One Governed Workflow End to End

1

Bound execution

Configure

Route requests with PreToolUse;
authorize MCP servers centrally

Exercise

Run ordinary, publication, protected-path, and blocked-server fixtures

Observe

See allow, ask, deny, and an explicit owner route

2

Bind evidence

Configure

Add scoped instructions and an
Actions workflow

Exercise

Open a pull request; push again
after the first check run

Observe

Tests and scans attach to the latest
commit

3

Enforce acceptance

Configure

Require CODEOWNERS, current
checks, and resolved threads

Exercise

Try to merge after a new commit or
with an open thread

Observe

GitHub blocks merge until every
condition is current



Start with one workflow. Expand after the team can reproduce every route, result, and recovery.

Runtime and context

[Enterprise Copilot policies](#)

Availability, scope, and policy precedence

[Hooks configuration reference](#)

PreToolUse payloads, decisions, timeouts, and failure behavior

[Repository custom instructions](#)

Repository-wide and path-scoped guidance

Evidence and merge authority

[Workflow syntax for GitHub Actions](#)

Triggers, jobs, permissions, and revision-bound checks

[About rulesets](#)

Required checks, pull requests, protection, and bypass visibility

[About code owners](#)

Path ownership and qualified review requests

MCP governance

[MCP server usage in your company](#)

Enterprise enablement, registries, and policy boundaries

[Configure the enterprise MCP allowlist](#)

Allowed and denied server evaluation

[Add MCP servers for Copilot CLI](#)

Repository declarations, tools, and loading precedence

Related talks

[GitHub Copilot Hooks](#)

Hook lifecycle mechanics and runtime contracts

[PR Trust Stack](#)

Evidence, review, and protected merge patterns



Useful Autonomy, Clear Authority

ONE CHANGE KEEPS ITS AUTHORITY CHAIN FROM REQUEST TO PROTECTED MERGE



Tool request

Copilot proposes an action

PreToolUse returns allow, ask, or deny



MCP access

The repository proposes a server

Enterprise policy decides whether it may run



Current evidence

Actions publishes reproducible results

Checks rerun against the latest commit



Protected merge

CODEOWNERS selects the qualified reviewer

The ruleset requires current approval and checks

Choose one consequential workflow: who may act, what proves the result, and who may approve it?